# Fashion Search Engine

Part 1: Text Processing and Exploratory Data Analysis

Marc Mallol Èric Dürr Oriol Pont

Information Retrieval and Web Analytics, 2026

## 1 Introduction

This report covers Part 1 of our fashion search engine: text processing and exploratory data analysis of a catalogue of 28,080 products from Flipkart, an Indian e-commerce site. Each product has a title, an optional description, a brand, a category and sub-category, a list of product details (fabric, sleeve, colour, ...), a seller, prices, a discount, an average rating, a stock flag and a URL. The goal of this part is to turn the raw catalogue into clean records that the later parts index (Part 2), rank (Part 3) and serve through a RAG web application (Part 4).

Section 2 describes the data preparation: the text pre-processing pipeline, the fields kept for each result, and how categorical and numerical fields are handled. We check each decision against the two validation queries in `validation_labels.csv`, *women full sleeve sweatshirt cotton* and *men slim jeans blue*. Section 3 explores the text and the products of the catalogue, and Section 4 summarises what this means for the search engine.

The code is at `https://github.com/uripont/fashion-search-engine`, tag `IRWA-2026-part-1`. The `fashion_search` package holds the reusable code, and `notebooks/part_1.ipynb` runs it and produces every number and figure in this report. The README explains how to reproduce them.

## 2 Data preparation

The code is in the `fashion_search` package: `text.py` turns text into search terms and `corpus.py` builds the product records. Section 2 of `notebooks/part_1.ipynb` runs them and holds the evidence for each decision. Together they turn each of the 28,080 products into one record that keeps every field a result must show, with numbers parsed, plus one list of terms per searchable field. The records are saved to `data/processed/products.parquet`, which the next parts of the project read.

### 2.1 Text pre-processing

The same function, `build_terms`, processes every text field and every query, so query terms and document terms always match. It applies, in order:

1. **Repair and normalize.** 91 descriptions contain UTF-8 text that was decoded as Latin-1 (`â\x80\x99` instead of an apostrophe), and 6,817 titles contain non-breaking spaces. We undo the former, fold accents and special spaces to plain ASCII and lowercase everything.

2. **Unify spelling variants.** The catalogue writes the same garment as *t-shirt*, *t shirt*, *tshirt* and *tee*; all become *tshirt*. *v-neck* and *v neck* become *vneck*, and glued words such as *slimfit* are split (*slim fit*).

3. **Possessives and hyphens.** *women's* becomes *women*, so it matches the query word; hyphenated words are joined (*round-neck* becomes *roundneck*).

4. **Remove punctuation.** Every other symbol becomes a space. Removing it instead would glue words that the source writes without a space after a full stop (*cotton.Designed*).

5. **Tokenize** with NLTK's `word_tokenize` [1].

6. **Remove stop words, numbers and single letters,** using NLTK's English stop word list. Gender words (*men*, *women*) are not in it and are kept, since both validation queries depend on them.

7. **Stem** with the Porter stemmer (*sleeves* and *sleeve* become *sleev*). Lemmatization would give readable terms, but it needs part-of-speech tags to be accurate and merges fewer variants.

### 2.2 Fields returned for each result

All fourteen fields a result must show are kept in their original form: `pid` (the key used for evaluation), `title`, `description`, `brand`, `category`, `sub_category`, `product_details`, `seller`, `out_of_stock`, `selling_price`, `discount`, `actual_price`, `average_rating` and `url`. The term lists are added next to them, never in their place, so a result page can show the original text. Empty strings are stored as missing values, so a result can leave out what a product does not have. Many fields are often missing (table 1); most importantly, 40% of products have no description.

| Field | Present | Field | Present |
|---|---|---|---|
| description | 60.3% | average_rating | 91.9% |
| brand | 92.8% | discount | 97.0% |
| seller | 94.1% | actual_price | 97.2% |

Table 1: Fields that are often missing. `selling_price` is missing in only 2 products; the other seven fields are present in every product.

### 2.3 Categorical fields

There are three ways to handle `category`, `sub_category`, `brand`, `product_details` and `seller`:

- **Merge them into one text field.** It is simple, and a query matches whatever field a word is in. But a match in a generic field counts as much as a match in the title, and the field's statistics are mixed together. For example, *clothing* would appear in 97% of products and say nothing.

- **Index them as separate fields.** The ranking can weight each field differently (a title match is stronger evidence than a description match) and each keeps its own term statistics. It costs one index per field and more complex queries.

- **Use them as filters.** Exact values, chosen by the user or detected in the query, restrict the results without affecting the text score. This only works for fields with few, well-defined values.

We chose by looking at where the words of the two validation queries appear in the products judged relevant (fig. 1). *Cotton* never appears in a title: it is only in the description or in the Fabric detail. *Full* and *sleeve* are in the Sleeve detail of every relevant product. *Blue* and *men*

are in Color and Ideal For. No query word ever appears in brand, category or sub-category. Because queries are conjunctive, a product is found only if *every* query word is indexed: with titles and descriptions alone, 11 of the 13 relevant products for query 1 are found, and adding the product details finds all 13.

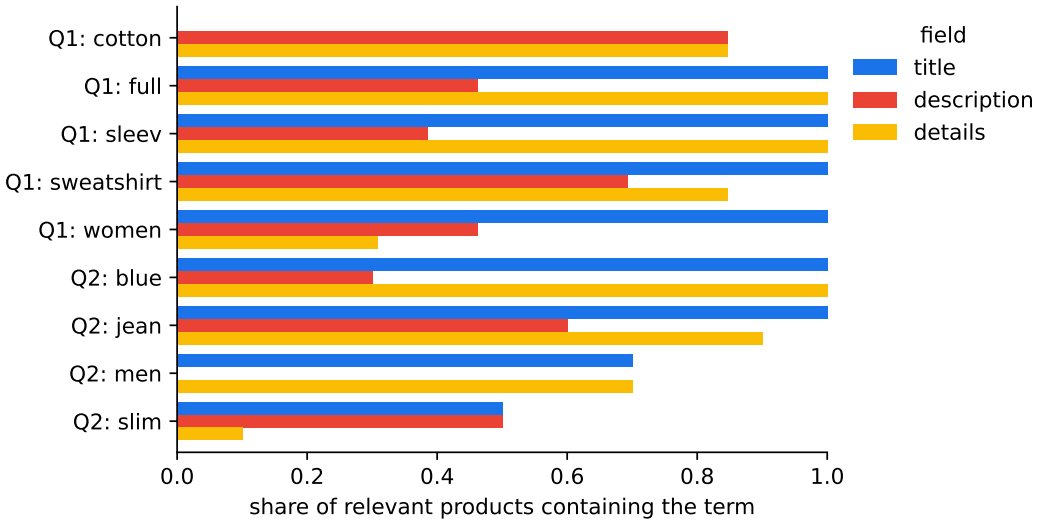



Figure 1: Share of relevant products for each validation query that contain each (stemmed) query term, by field. Brand and category contain none of them.

Our decision, field by field:

- **Title, description, product details and brand are indexed as separate text fields.** A plain inverted index can still use all of them together (a document's terms are the union of the four), while later ranking can weight them separately. From `product_details` we index the *values* (*Cotton Blend*, *Full Sleeve*, *Slim*) but not the keys, which repeat in almost every product. We drop *Style Code*, an internal product code. Brand is distinctive (324 brands; the most common has only 3.6% of products), so a query naming a brand should find it, although many brand names are truncated in the source data (section 3.2).
- **Category and sub-category are filters.** There are only 4 categories, and 96.6% of products are *Clothing and Accessories*; there are 24 sub-categories, and *Topwear* alone has 54.5%. As text they would add an almost constant word to every product; as filters (*Footwear* only) they are precise.
- **Seller is a filter and is displayed, not searched.** Its 534 values are company names that do not describe the product, so they would match unrelated queries.

The second validation query shows the limits of conjunctive queries: whatever fields are indexed, only 6 of its 10 relevant products contain *slim*, since the others are *tapered* or *super skinny* jeans that the judges still found relevant. Some identical products are also judged differently, so the labels contain some noise. We return to this in Part 2.

## 2.4 Numerical fields

`selling_price`, `actual_price`, `discount` and `average_rating` arrive as text (`"2,999"`, `"69% off"`, `"3.9"`). We parse them into numbers and keep missing values as missing, not as zero, which would make a product look free or badly rated (table 2). `out_of_stock` is already a

| Field | Min | Median | Max | Missing |
|---|---|---|---|---|
| selling_price (INR) | 99 | 545 | 7,999 | 2 |
| actual_price (INR) | 150 | 1,199 | 12,999 | 777 |
| discount (%) | 1 | 53 | 87 | 855 |
| average_rating | 1.0 | 3.8 | 5.0 | 2,261 |

Table 2: Numerical fields after parsing.

boolean. Two checks confirm the parsing: no selling price is above its actual price, and every parsed discount is within one percentage point of the one implied by the two prices.

These fields are **not indexed as text terms**. A term only helps if users type it, and nobody searches for *921*. Each price would also become its own rare term, so TF-IDF would give it a high weight, and as text it could not express *under 1,000*. Kept as numbers, they serve three purposes: filters (price range, minimum rating, in stock only), signals for our own ranking score in Part 3 (a well-rated, discounted, available product is a better result), and information on the results page.

# 3 Exploratory data analysis

Section 3 of the notebook produces every figure below.

## 3.1 Text

**Length.** Titles are short and uniform: 6.5 words on average (median 6, maximum 38). Descriptions are longer and far more variable: 29.8 words on average, but 40% of products have none, so the median is only 10, and the longest has 623 words (fig. 2). Titles are noun phrases, not sentences. Splitting descriptions into sentences with NLTK gives 3.1 sentences of 15.7 words on average (medians 2 and 14): short marketing blurbs rather than long text. This is approximate, because some descriptions omit the space after a full stop, and those sentence pairs count as one. Since every product has a title and only 60% have a description, the title is the one field we can rely on for every product.

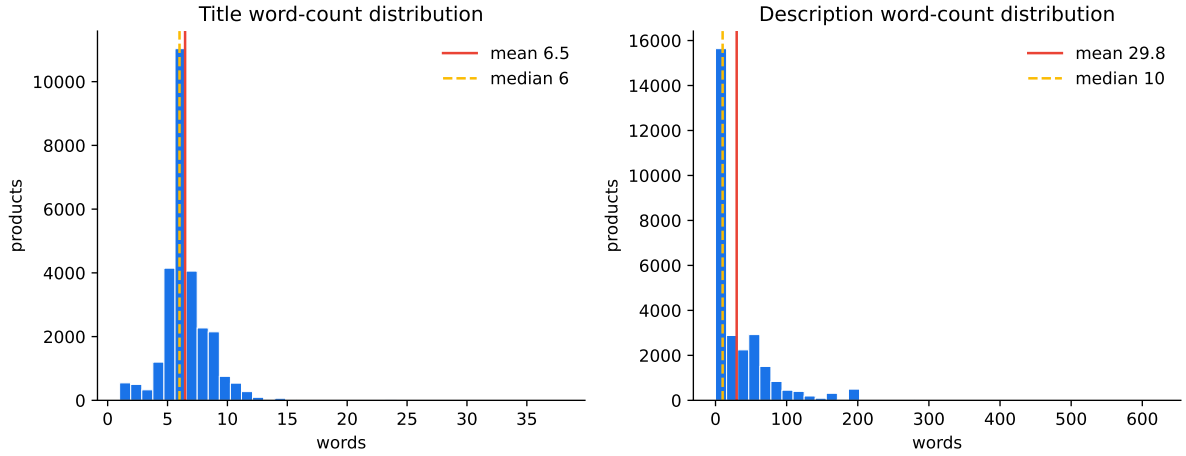



Figure 2: Words per title and per description (raw text).

**Vocabulary.** Counting lowercase whitespace-separated words, titles use 953 distinct words and descriptions 10,250. Pre-processing reduces them to 652 (−32%) and 4,933 (−52%) terms

(fig. 3); product details add 3,604 terms and brands 388. Titles share a tiny vocabulary because they follow a template (pattern, gender, neck, colour, garment, as in *Printed Women Round Neck Black T-Shirt*), so many products have nearly identical titles.

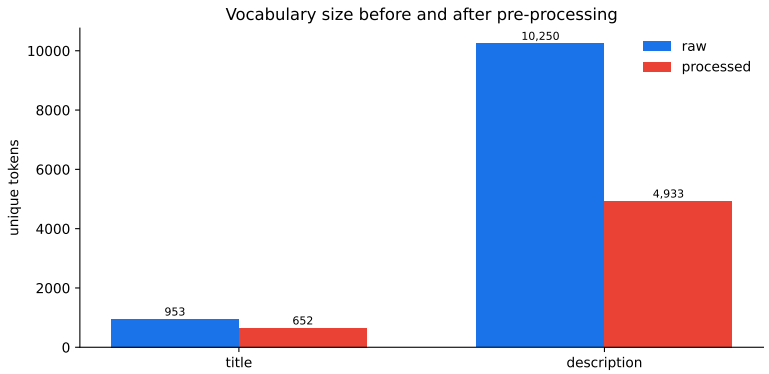



Figure 3: Vocabulary size per field, before and after pre-processing.

**Frequent terms.** The catalogue is dominated by t-shirts: *tshirt* is the most frequent term in titles and descriptions (28,256 occurrences), followed by *women* (17,569), *men* (16,831), *neck*, *print* and *cotton* (figs. 4 and 5). Several words of the validation queries (*women*, *men*, *cotton*, *sleev*, *blue*) are among the most frequent terms. They appear in so many products that they barely tell them apart, so TF-IDF will give them low weight and the rarer garment words (*sweatshirt*, *jean*) will drive the ranking.

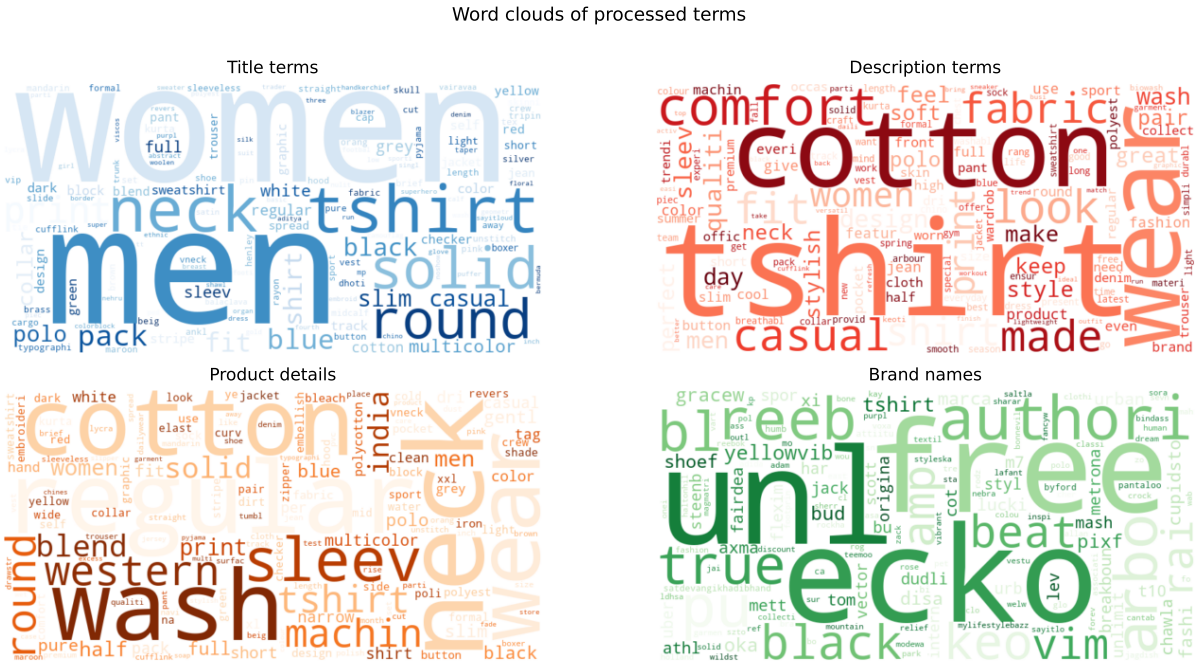



Figure 4: Most frequent terms per field, after pre-processing.

**Named entities.** We ran spaCy's `en_core_web_sm` model [2] on a random sample of 2,000 titles. It finds an entity in 1,402 of them, 1,772 in total (ORG 930, PERSON 421, CARDINAL 378), but nearly all are wrong (fig. 6): *Solid Women* is tagged as a person, whole titles such as *Printed Women Round Neck Black T-Shirt* as organisations, *Kurta* as a place, and the

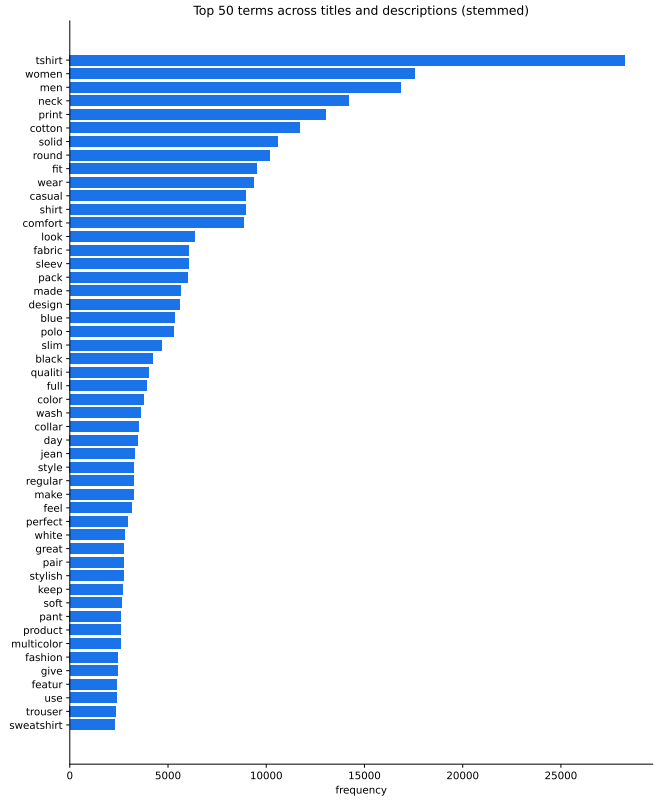



Figure 5: The 50 most frequent terms in titles and descriptions (stemmed).

CARDINAL entities are pack sizes. The model, trained on news text, reads capitalised product titles as names. The real entities, brands, already have their own field, so we do not use entity recognition for indexing.

## 3.2 Catalogue

**Categories.** 96.6% of products are *Clothing and Accessories*; the rest are mostly *Footwear*, with only a handful of *Bags, Wallets & Belts* and *Toys* (fig. 7). *Topwear* alone holds 54.5% of products, followed by *Bottomwear* and *Winter Wear*. Some sub-category values are noisy, with a brand glued to a category name (*Uber Urban Clothing and Accessories*) or the same value in two spellings (*SUNSHOPPING* and *Sunshopping Bags, Wallets & Belts*); they are rare, but a category filter has to account for them.

**Brands and sellers.** Both have a long tail: 324 brands and 534 sellers, and the largest of each (seller *RetailNet*, about 1,400 products, 5.3%; brand *ECKO Unl*, 3.6%) holds only a small share, and the top 20 of each cover about 40% of products (fig. 8). Brand names are truncated in the source data (*Pu*, *True Bl*, *V*), while seller names are complete (*ArvindTrueBlue*). A query for the full brand name will therefore not match the brand field, so brand matches will be less useful than its distinctiveness suggests.

**Prices and discounts.** Prices are right-skewed (fig. 9). The median selling price is INR 545 (half of products between 390 and 820), while the median actual price is INR 1,199. Almost every product is discounted, and heavily: the median discount is 53%, half of products are between 40% and 63% off, and the maximum is 87%. The actual price therefore acts as a reference price; the selling price is what the user pays and the one to filter on.

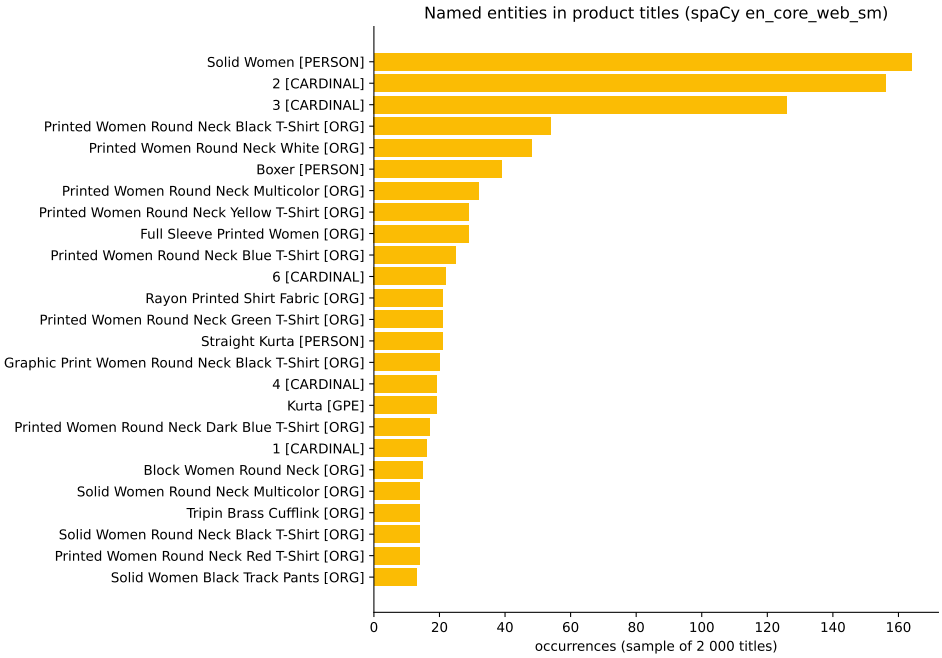



Figure 6: The 25 most frequent named entities in a sample of 2,000 titles.

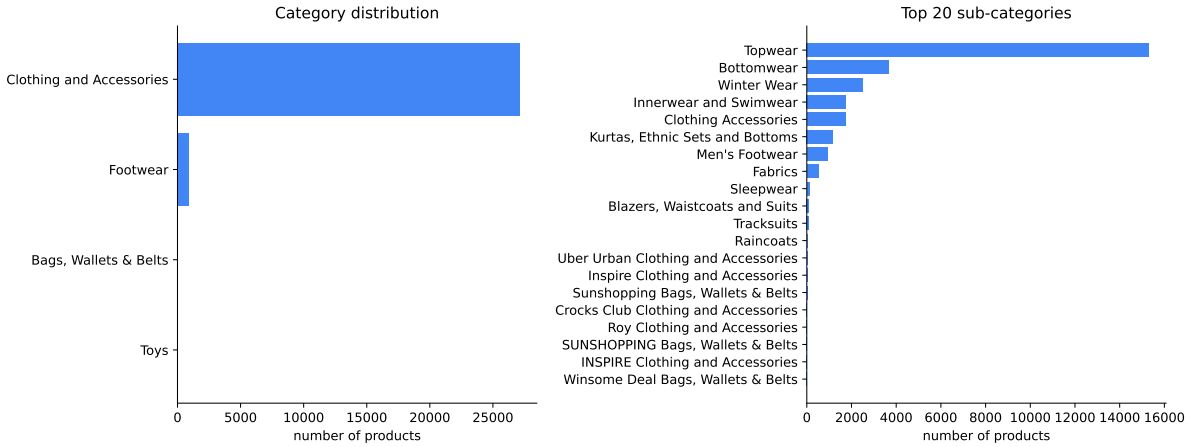



Figure 7: Products per category and per sub-category (top 20).

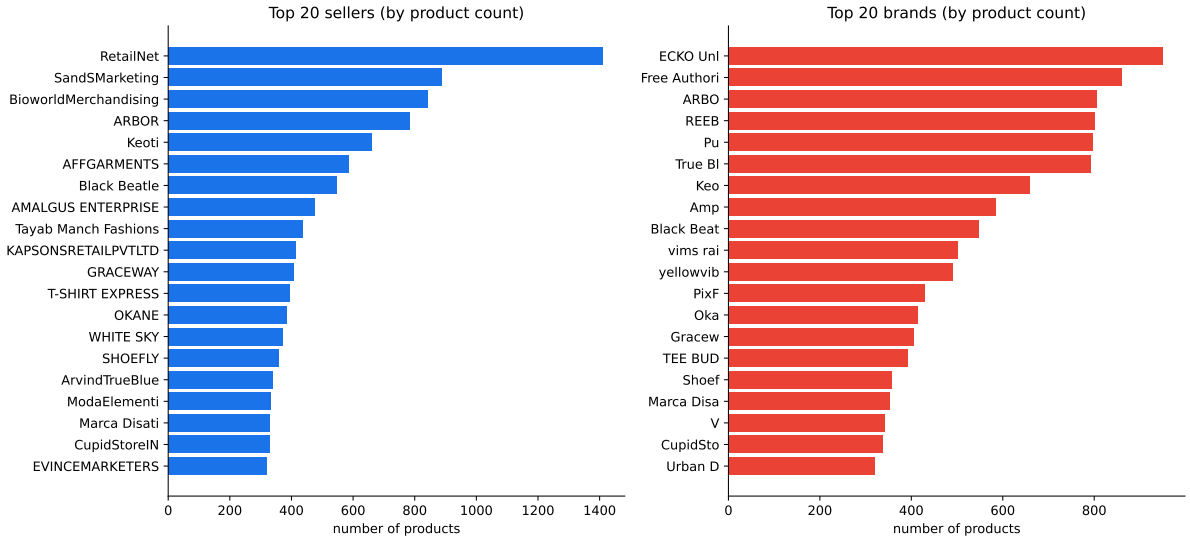



Figure 8: The 20 sellers and brands with the most products.

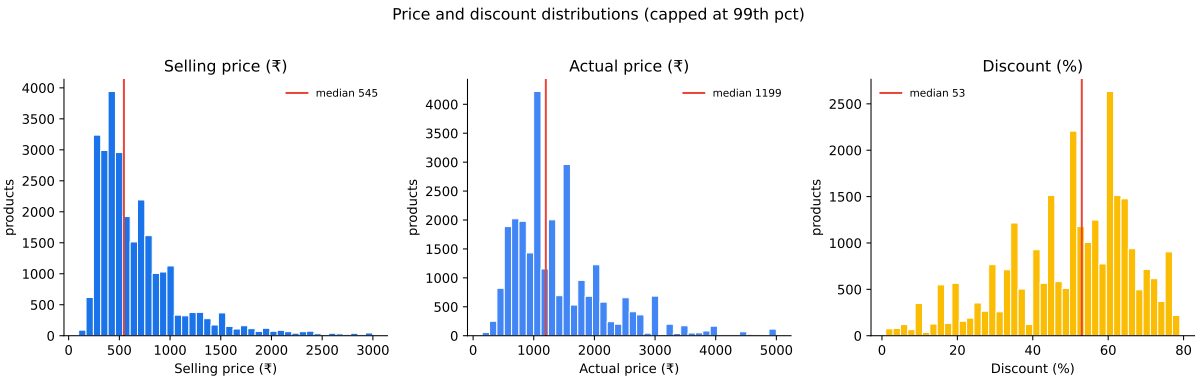



Figure 9: Selling price, actual price and discount, capped at the 99th percentile.

**Ratings and stock.** The median rating is 3.8 (mean 3.63, half of products between 3.2 and 4.1); 16.8% of rated products are below 3, only 1.3% have exactly 5.0, and 8.1% of products have no rating (fig. 10). Only 5.9% of products are out of stock, but these should not be ranked above available ones, so stock will act as a filter or a penalty.

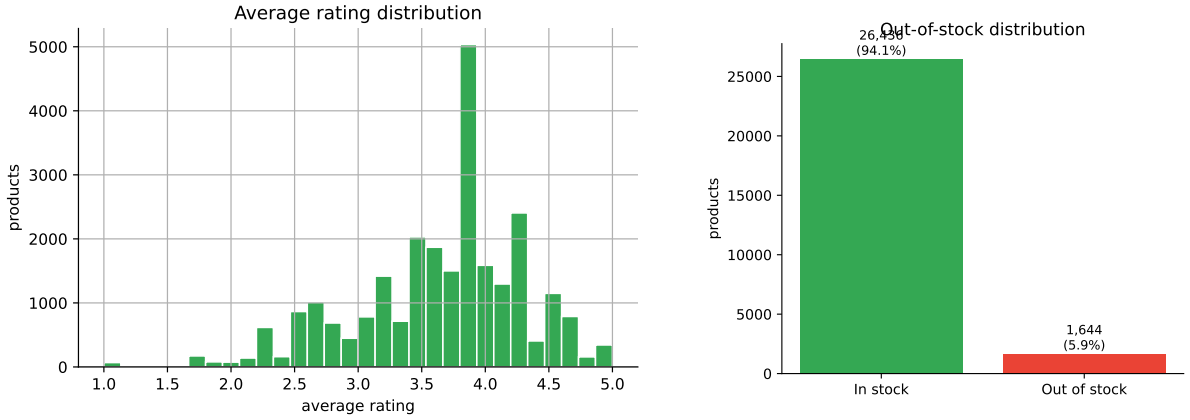



Figure 10: Average rating (left) and stock (right).

**Rankings by a single field.** Sorting by one field shows why none of them works alone as a ranking. The top products by rating are all ties at 5.0, and since the dataset has no review counts we cannot tell whether a 5.0 comes from one review or from thousands. The most discounted products are cheap t-shirts at 84–87% off (INR 128 instead of 998), and the most expensive are suits and jackets at INR 12,999. Rating, discount and price are useful signals on top of text relevance, which is how Part 3 will use them, but not as rankings by themselves.

# 4 Conclusions

The prepared corpus keeps every field a result must show, with numbers parsed and missing values kept as missing, plus one list of terms for each of the four searchable fields. Our main decisions are:

- **One pipeline for documents and queries.** Repairing and normalising the text, unifying spelling variants, removing stop words and stemming shrink the title vocabulary by 32% and the description vocabulary by 52%, and make query words such as *women's*, *t-shirt* or *sleeves* match the way products write them.
- **Title, description, product details and brand are searched; category, sub-category and seller are filters.** Product details are needed: without them, 2 of the 13 relevant products of the first validation query miss a query word. Category barely varies (96.6% of products share one value) and never contains a query word, so it only works as a filter.
- **Numbers are not terms.** Prices, discount, rating and stock serve as filters, as signals for our ranking score in Part 3, and as information on the results page.

The analysis also shows what to expect when indexing and ranking. Titles are short and follow a template, and 40% of products have no description, so many products have nearly identical text, and document length varies mostly with the description. The most frequent terms (*tshirt*, *women*, *men*, *cotton*) include most words of the validation queries, so TF-IDF will rank mainly by the rarer garment words. No numeric field ranks well on its own: the best-rated products tie at

5.0 with no review counts to separate them, and almost every product is discounted. Finally, the data has problems that the next parts must handle: truncated brand names, noisy sub-category values, and validation labels that do not always contain the literal query words (only 6 of the 10 relevant products for *men slim jeans blue* contain *slim*). An off-the-shelf entity recogniser mislabels most product titles, so we rely on the structured fields instead.

## Use of AI tools

We used Claude (Anthropic), through Claude Code, to review the notebook and this report. It helped write the code production and debugging, and helped draft this report from the notebook outputs. We checked every number against the notebook outputs and reviewed and edited the text ourselves.

## References


[1] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O'Reilly Media, 2009.

[2] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in python, 2020.